新手入門

一律下一步

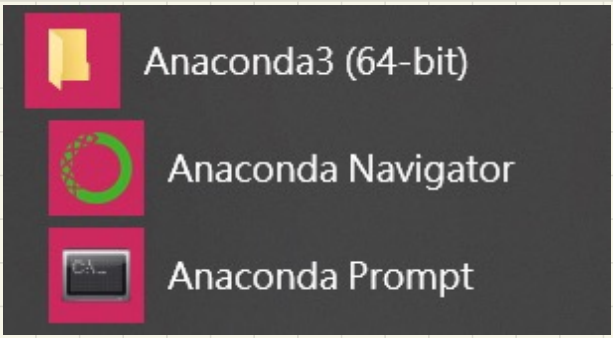
1. 下載 Visual Studio Code

2. 下載 Anaconda

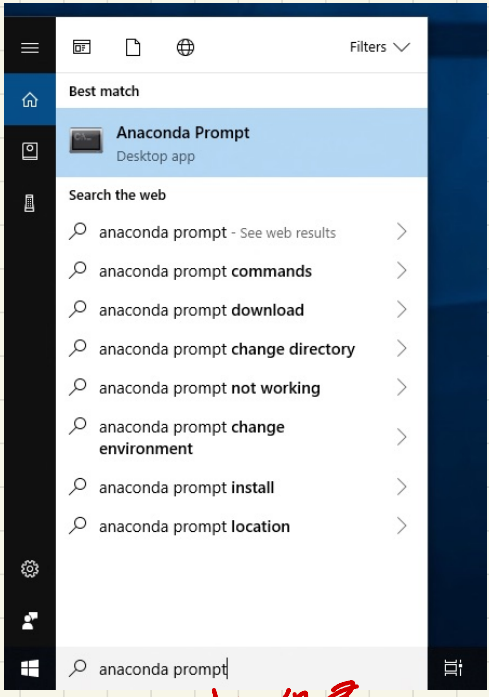
3. 左下角開始欄打開 Anaconda Prompt

※不是 Anaconda Powershell Prompt

(開始創建環境)



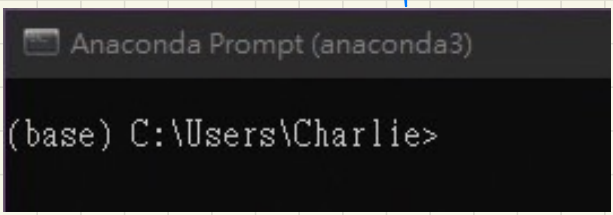
※通常在 Anaconda3 底下的資料夾



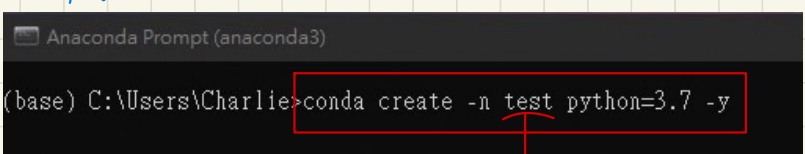
※找不到就搜尋

建立環境

1. 打開 Anaconda Prompt

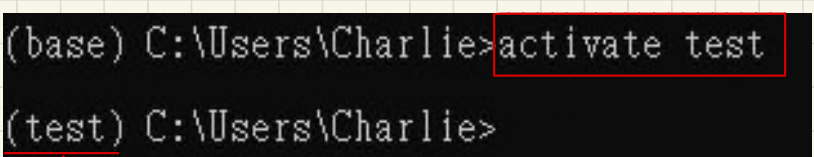


2. 輸入紅框內容



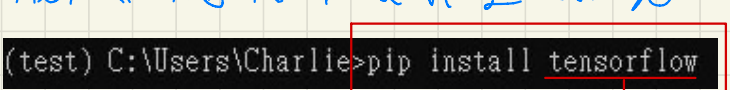
名字可自定義

3. 輸入紅框，進入已創建好的環境



下一行開頭有環境名稱，代表成功進入環境

4. 輸入紅框，加入套件至環境



※ 跑完 tensorflow，再跑右邊三個套件，指令一樣，pip install numpy

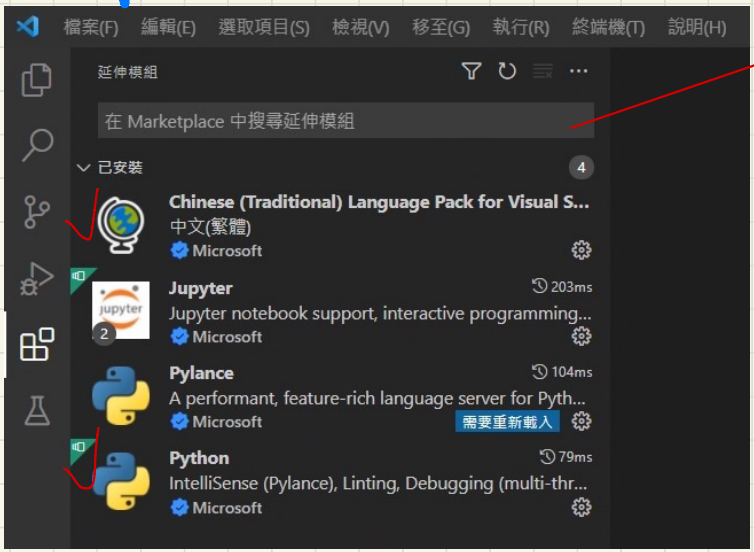
⋮

以此類推

numpy
opencv-python
matplotlib

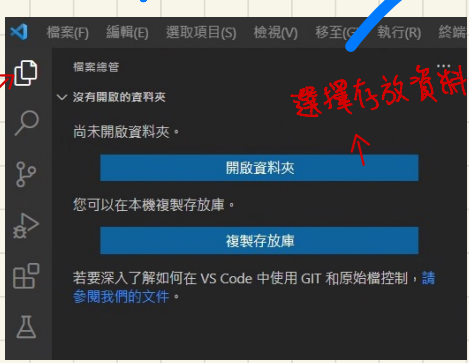
Vistual Studio Code Hello World

Step 1



搜尋輸入打包項目名稱

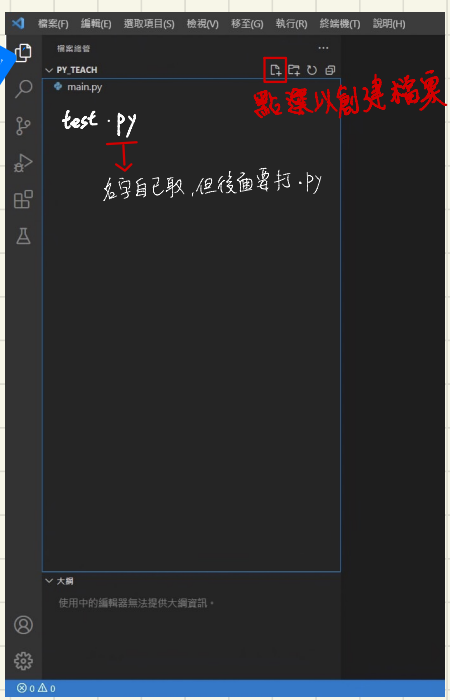
Step 2



點第一個

選擇存放資料夾

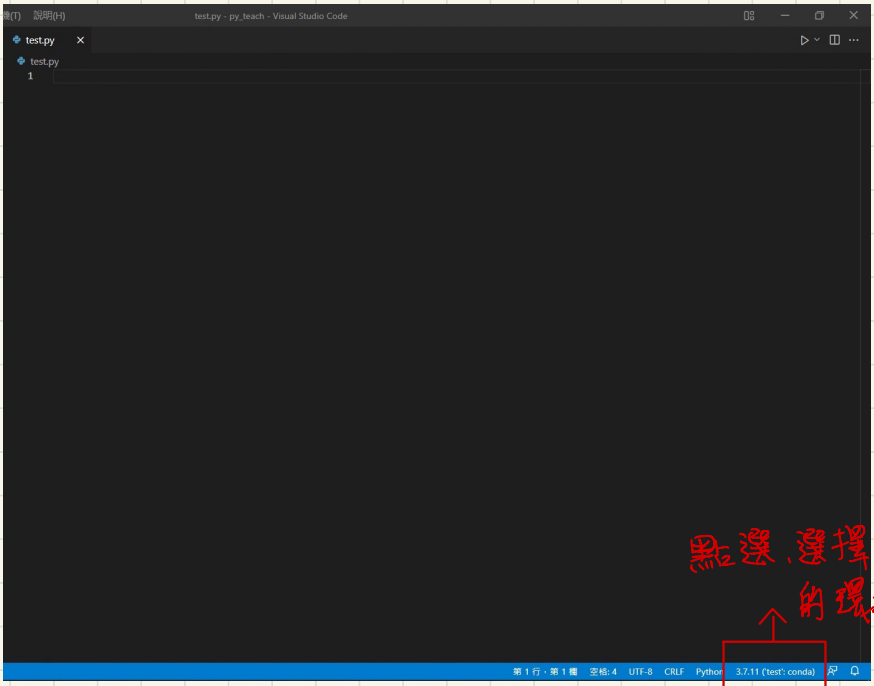
Step 3



點選以創建檔案

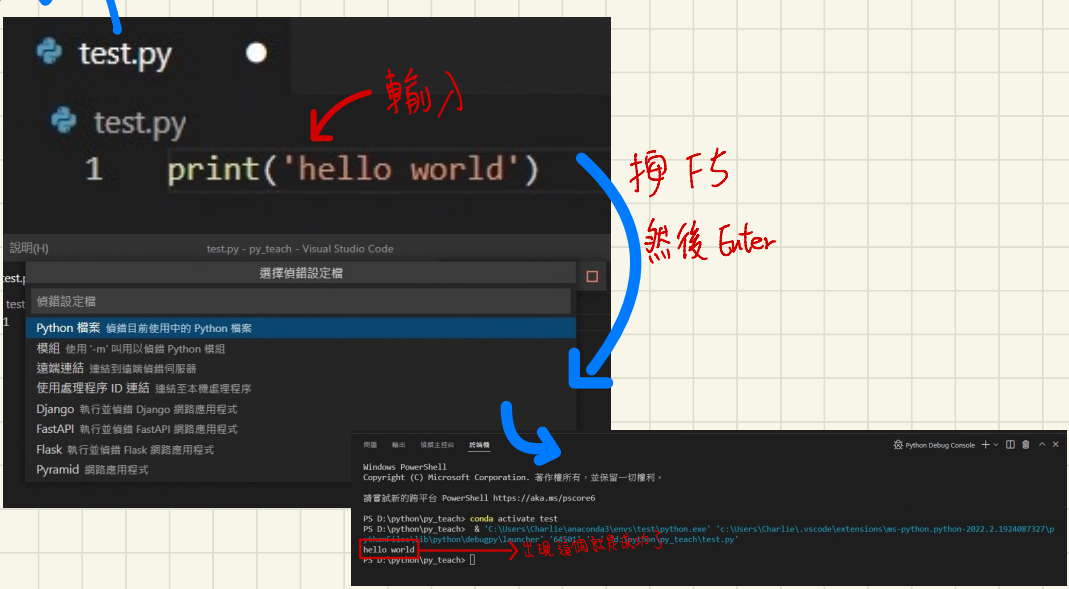
名字自己取，但後面要打.py

Step 4



點選、選擇剛創建的環境名稱

Step 5



神經網路基礎(手寫辨識)

會給兩個壓縮檔

- MNIST_Dataset.rar
- programming_code.rar

解壓縮

打開資料夾方式，跟前面VS的教學一樣，選擇資料夾，選擇解壓縮的資料夾。

開啟方式

(放在同一個資料夾)

(資料集)

MNIST_Dataset

- (訓練集) MNIST_Training
  - 0
  - 5
  - 9
  - (內容)
- (驗證集) MNIST_Validation
  - 0
  - 5
  - 9
  - (內容)

(程式)

programming_code

- mnist.py (主程式)
- gradcam.py (熱力圖)

資料集：用於訓練神經網路，神經網路會抓取資料集的特徵，讓神經網路知道訓練物為何物。通常有訓練集、驗證集、測試集。

訓練集：輸入至神經網路做訓練，讓神經網路抓取特徵值。

驗證集：通常是從訓練集分割出的數據，驗證集做為神經網路訓練完畢時，輸入至模型，來評估模型訓練成效。

測試集：通常是沒有輸入至神經網路的資料，來測試模型在未知的資料下，辨識的成果如何。

主程式：通常是輸入 Normalization、Data Augmentation、Model etc.

熱力圖：來判斷神經網路是根據哪個部位做辨識，這樣可以更加了解神經網路的訓練思為。

程式詳解 打開 MNIST.py

```python
# 輸入函示庫
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow import keras
import tensorflow.keras.backend as K
from tensorflow.keras import layers, models
from tensorflow.keras.models import Sequential
from tensorflow.python.keras.applications.resnet import preprocess_input
import cv2
from gradcam import get_img_array , make_gradcam_heatmap , save_and_display_gradcam
```

import 套件名稱

匯入熱力圖程式

整行

```python
# 讀取資料集
train_data_dir = "./MNIST_Dataset/MNIST_Training"  # 也可以輸入絕對路徑
validation_data_dir = "./MNIST_Dataset/MNIST_Validation"

# 參數設定
batch_size = 256  # 批量
img_height = 28  # 照片高度
img_width = 28  # 照片寬度
array = 1  # 維度
activate = 'relu'  # 激勵函數
lr = 0.0001  # Learning_rate 學習率
num_classes = 10  # 分類數量
epochs = 30  # 訓練次數
```

（訓練集）

（驗證集）

絕對路徑

ex. D:/MNIST_Dataset/MNIST_Train

以參數都可以做設定

4 各參數的意思就自行搜尋，影片都有答案，不懂在問我

搜尋關鍵字：1. Batch Normalization

2. 圖像維度

3. Activation Function 激勵函式

4. Learning Rate 學習率

5. Epochs

```python
# 訓練集與測試集設定
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    color_mode="grayscale"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    validation_data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    color_mode="grayscale"
)
```

驗證集比例
資料定位
圖像尺寸
Batch Size 大小
顏色模式

```python
# 確認尺寸
for image_batch, labels_batch in train_ds:
    print(image_batch.shape)
    print(labels_batch.shape)
    break
```

確認圖像以及標籤的尺寸

```python
# 確認類別
class_names = train_ds.class_names
print(class_names)  # ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'] 共十個類別
```

確認類別名稱，若要確認數量

print(len(class_names))

```python
# 定義模型
model = Sequential([
    # layers.Rescaling( 正規化 , input_shape = (長 , 寬 , 維度)
    layers.Rescaling(1./255, input_shape=(img_height, img_width, array)),
    # 卷積層 (神經元數量 , 尺寸 , padding = 'valid' or 'same' , strides = 步數 , activate = 激勵函數 ('sigmoid' or 'tanh'))
    layers.Conv2D(32, 3, padding='valid', strides=(1, 1), activation=activate),
    # 池化層 (pool_size = 池化尺寸)
    layers.MaxPooling2D(pool_size=(2, 2)),
    # Dropout = 隨機關閉神經元 (數量)
    layers.Dropout(0.2),
    # 扁平層
    layers.Flatten(),
    # full connection layer 全連接層
    layers.Dense(32, activation=activate),
    layers.Dense(num_classes, activation='softmax')
])
```

(輸入大小設定)

激勵函數

神經元數量、padding、stride、activation etc. 都可以更改(設定)

需自行了解的東西，不懂的先問我

關鍵字 1. Normalization 正規化

2. Convolution layer 卷積層 (計算方式也要了解)

3. Convolution Stride

4. 神經元數量

5. Pooling layer 池化層 (有不同種類) (計算方式也要了解)

6. Dropout

7. Flatten layer

8. Dense layer

9. Full Connected layer 全連接層

設定學習率

```python
# 訓練模型配置
model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),              # 設定優化器以及 Learning_rate
              loss=keras.losses.SparseCategoricalCrossentropy(
                  from_logits=True),  # 設定 loss 計算方式
              metrics=['accuracy'])                                           # 設定 accuracy 計算方式
model.summary()
```

可以試著了解 optimizer 和 loss function 是什麼.

關鍵字: 1. 優化器 optimizer

2. 損失函數 loss function

```python
# 進行模型訓練
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

model.fit()
匹配已資料集、驗證集以及 epochs
訓練次數

模型與資料集做結合（match）和設定訓練次數

history 做為後續顯示圖像的參數

```python
# 輸出圖形以便觀察訓練成果
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')

plt.show()
```

基本上不用動

```python
# 選擇測試資料做測試
img_path = "./Test_data/5.jpg"  # 可以選擇絕對路徑
img = tf.keras.utils.load_img(
    img_path, target_size=(img_height, img_width),
    grayscale=True
)
img_array = tf.keras.utils.img_to_array(img)
img_array = tf.expand_dims(img_array, 0)  # Create a batch

predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])
print(score)
print(
    "這個可能是 {} 的機率是 {:.2f} %."
    .format(class_names[np.argmax(score)], 100 * np.max(score))
)
```

你要辨識的數字影像